

리눅스 기반 - Docker 동작 원리

남궁명수

Docker 컨테이너는 가상 컴퓨터가 아니라, 리눅스 커널이 격리하고 자원 사용량을 제한한 프로세스다.

컨테이너 안에서 Nginx를 실행해도 결국 호스트 리눅스 입장에서는 하나 이상의 일반 프로세스이다.

호스트 리눅스

- ├ PID 1000: dockerd
- ├ PID 1500: containerd
- ├ PID 2100: 컨테이너 A의 nginx
- └ PID 2200: 컨테이너 B의 mysql

다만 각각의 프로세스에 다음 리눅스 기능을 적용한 것이다.

- Namespace → 무엇을 볼 수 있는가?
- cgroup → 자원을 얼마나 사용할 수 있는가?
- OverlayFS → 어떤 파일시스템을 보는가
- Capability → root 권한 중 무엇을 사용할 수 있는가?
- Seccomp/LSM → 어떤 시스템 콜과 행위를 허용할 것인가?

[1] 가상머신과 컨테이너의 차이

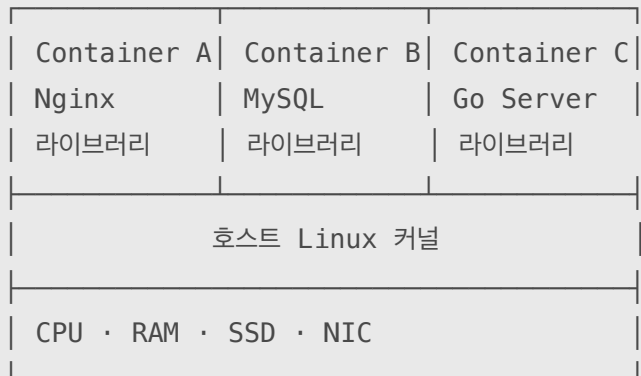
[가상머신]

가상머신은 게스트 운영체제와 게스트 커널을 따로 실행한다.



[Docker 컨테이너]

컨테이너에는 별도의 리눅스 커널이 없다. 모든 컨테이너가 호스트 리눅스 커널을 공유한다.



컨테이너 안에 /bin/bash, /etc, /usr, 라이브러리 등이 있으니 운영체제처럼 보이지만,

정확히는 **프로그램 실행에 필요한 사용자 공간 파일**이 들어 있는 것이다.

컨테이너에 들어 있는 것

- └ 실행 파일
- └ 공유 라이브러리
- └ 설정 파일
- └ 루트 파일시스템

컨테이너에 없는 것

- └ 독립적인 Linux 커널

그래서 컨테이너가 호스트와 다른 커널을 사용할 수는 없다.

[2] Docker의 전체 구성

일반적인 Linux Docker Engine 구조는 다음과 같다.

flowchart TD

```
A["docker CLI"] --> B["dockerd<br/>Docker daemon"]
B --> C["containerd<br/>컨테이너 수명주기 관리"]
C --> D["containerd-shim<br/>실행 중 컨테이너 연결 유지"]
D --> E["runc<br/>Namespace·cgroup 설정"]
E --> F["Linux 커널"]
F --> G["격리된 컨테이너 프로세스"]
```

Docker daemon은 containerd를 통해 OCI 호환 런타임을 사용하며, 기본 런타임은 일반적으로 runc이다.

runc가 리눅스의 Namespace, cgroup 등의 기능을 이용해 실제 컨테이너 프로세스를 만든다.

[Docker CLI]

우리가 터미널에서 사용하는 명령어이다.

```
docker run nginx
docker ps
docker stop web
```

CLI가 직접 컨테이너를 만드는 건 아니다. Docker daemon에 API 요청을 보낸다.

```
docker CLI
  |
  | Docker API 요청
  ▼
/var/run/docker.sock
  |
  ▼
dockerd
```

[dockerd]

Docker 전체를 관리하는 데몬이다.

- 이미지 다운로드 및 관리
- 컨테이너 생성 요청
- 네트워크 생성
- 볼륨 관리
- 빌드 관리
- containerd에 실행 요청 전달

[containerd]

컨테이너의 실행 수명주기를 관리한다.

- 이미지와 컨테이너 관련 데이터 관리
- 컨테이너 생성·시작·중지·삭제
- OCI 런타임 호출
- containerd-shim 관리

[runc]

Docker가 컨테이너 프로세스를 실제로 생성하고 실행할 때 사용하는 저수준 컨테이너 런타임이다.

runc가 실제로 리눅스 커널 기능을 사용해 컨테이너 프로세스를 만든다.

대략 다음을 설정한다.

- Namespace 생성
- cgroup 설정
- Root filesystem 준비
- Capability 제한
- Seccomp 적용
- 컨테이너 시작 프로세스 실행

OCI Runtime Specification도 Linux 컨테이너를 구성하기 위해 Namespace, cgroup, Capability, LSM, 파일 시스템 격리를 사용한다고 정의한다.

[3] Namespace: 프로세스가 보는 세상을 분리

Namespace는 프로세스가 볼 수 있는 커널 자원을 격리한다.

실제로는 같은 호스트에 있지만, 컨테이너 프로세스가 자신만의 시스템을 가진 것처럼 보이게 한다.

[PID Namespace]

프로세스 번호 공간을 분리한다.

호스트에서 보면:

```
PID 3150 nginx
```

컨테이너 안에서 보면:

```
PID 1 nginx
```

같은 프로세스를 서로 다른 PID로 보는 것이다.

```
호스트 PID Namespace
└─ PID 3150: nginx
    |
    └─ 컨테이너 PID Namespace에서는 PID 1
```

컨테이너 안에서 ps를 실행하면 호스트의 모든 프로세스가 보이지 않고 해당 PID Namespace에 속한 프로세스만 보인다.

```
# 컨테이너 내부
ps aux
PID COMMAND
1 nginx
7 nginx
8 ps
```

하지만 호스트에서는 컨테이너 프로세스도 확인할 수 있다.

```
ps -ef
```

[Mount Namespace]

프로세스가 보는 마운트 구조를 분리한다.

호스트가 보는 /

```
├─ /home
├─ /var
└─ /usr
```

컨테이너가 보는 /

```
├─ /bin
├─ /etc
├─ /usr
└─ /app
```

컨테이너의 /는 호스트의 실제 /와 다른 루트 파일시스템이다.

`pivot_root()` 또는 비슷한 구성 과정을 이용해 컨테이너 프로세스의 루트 파일시스템을 전환한다.

[Network Namespace]

네트워크 관련 자원을 분리한다.

각 컨테이너는 자신만의 다음 자원을 가질 수 있다.

- 네트워크 인터페이스
- IP 주소
- Routing Table
- ARP/Neighbor Table
- 방화벽 규칙
- Socket과 Port 공간

호스트 Network Namespace

```
├─ 물리 NIC: enp3s0
├─ docker0
└─ veth-host
```

컨테이너 Network Namespace

```
├─ eth0
├─ lo
└─ 별도의 Routing Table
```

컨테이너 안에서 `eth0`가 보이지만, 이것은 일반적으로 물리 NIC가 아니라 **가상 Ethernet 인터페이스인 veth의 한쪽 끝**이다.

[UTS Namespace]

Hostname과 domain name을 분리한다.

```
docker run --hostname web-server ubuntu hostname
```

컨테이너 안에서는 `web-server`라고 나오지만 호스트 이름은 바뀌지 않는다.

[IPC Namespace]

프로세스 간 통신 자원을 격리한다.

- System V Shared Memory
- Message Queue
- Semaphore
- POSIX Message Queue

컨테이너 A의 공유 메모리가 기본적으로 컨테이너 B에서 보이지 않게 한다.

[User Namespace]

컨테이너의 UID와 호스트의 UID를 다르게 매핑할 수 있다.

컨테이너 내부 UID 0(root)

↓ 매핑

호스트 UID 100000

즉, 컨테이너 안에서는 root처럼 보여도 호스트에서는 일반 UID로 취급할 수 있다.

Docker의 Rootless 모드도 User Namespace를 이용해 Docker daemon과 컨테이너를 비root 사용자로 실행한다.

[Cgroup Namespace]

컨테이너가 보는 cgroup 구조를 격리한다.

실제 호스트 cgroup 전체 구조가 컨테이너에 그대로 노출되는 것을 줄여준다.

[4] cgroup: 자원 사용량 제한

Namespace는 무엇이 보이는지 제한하지만, CPU와 RAM의 실제 사용량을 제한하지는 않는다.

그 역할을 cgroup이 담당한다.

Namespace → 격리

cgroup → 제한과 측정

예를 들어 컨테이너에 CPU 1개와 RAM 256MiB 제한을 준다고 해보자.

```
docker run \  
  --cpus="1.0" \  
  --memory="256m" \  
  nginx
```

Docker가 해당 컨테이너 프로세스를 별도의 cgroup에 배치한다.

호스트

├ 일반 프로세스

└ Docker Container cgroup

├ nginx master

├ nginx worker 1

└ nginx worker 2

[CPU 제한]

컨테이너 전용 CPU가 생기는 것은 아니다.
호스트 스케줄러가 컨테이너 프로세스도 일반 태스크처럼 스케줄링한다.

Run Queue

```
├─ 사용자 프로세스 A
├─ 사용자 프로세스 B
├─ 컨테이너 Nginx
└─ 컨테이너 MySQL
```

cgroup은 다음과 같은 정책을 추가로 적용한다.

- CPU 사용 가중치
- 일정 시간 동안 사용할 수 있는 CPU 시간 제한
- 사용할 수 있는 CPU 코어 지정

```
docker run --cpus="0.5" nginx
docker run --cpuset-cpus="2,3" nginx
```

--cpus="0.5"는 물리 CPU 반 개를 만들어 주는 게 아니라,
일정 기간 기준으로 CPU 시간 사용량을 제한하는 방식이다.

[메모리 제한]

컨테이너 프로세스가 malloc()을 호출하면 별도의 컨테이너 RAM에서 할당되는 게 아니다.

```
컨테이너 프로세스
  │ malloc()
  ▼
호스트 Linux 커널
  │ Page Fault
  ▼
호스트의 물리 페이지 할당
  │
  ▼
컨테이너 cgroup 사용량으로 기록
```

물리 메모리는 여전히 호스트 커널의 Buddy Allocator 등이 관리한다.
cgroup은 해당 물리 메모리 사용량을 컨테이너별로 계산하고 제한한다.

메모리 제한을 넘으면:

- 메모리 회수가 시도될 수 있음
- 할당이 지연되거나 실패할 수 있음
- 해당 cgroup 내부에서 OOM 처리가 발생할 수 있음
- 컨테이너 프로세스가 종료될 수 있음

[PID 제한]

컨테이너가 무한히 프로세스를 생성하는 것을 막을 수 있다.

```
docker run --pids-limit=100 nginx
```

이는 Fork Bomb 같은 문제의 영향을 제한하는 데 도움을 준다.

포크 폭탄(Fork Bomb)

컴퓨터 프로그램이 계속해서 자신을 복제하여 시스템 자원을 모두 써버리는 공격이다.

이로 인해 컴퓨터가 느려지거나 멈추게 된다.

이를 막으려면 Linux 시스템에서 사용자별 프로세스 생성 수를 제한해야 한다.

[5] Docker Image와 OverlayFS

[Image란?]

Docker Image는 컨테이너를 실행하기 위한 **읽기 전용 파일시스템과 실행 설정의 묶음**이야.

Docker Image

├ 실행 파일

├ 공유 라이브러리

├ 설정 파일

├ 애플리케이션 코드

├ 기본 실행 명령

└ 환경변수 등의 메타데이터

Image는 여러 개의 읽기 전용 Layer로 구성된다.

```
FROM ubuntu
```

```
RUN apt-get update
```

```
RUN apt-get install -y nginx
```

```
COPY index.html /var/www/html/
```

개념적으로는 다음과 같이 쌓인다.

Layer 4: index.html

Layer 3: nginx 설치 파일

Layer 2: apt 관련 변경 파일

Layer 1: Ubuntu 기본 파일

[컨테이너 실행 시]

컨테이너를 실행하면 Image Layer 위에 컨테이너 전용 쓰기 Layer를 추가한다.

Container Writable Layer ← 컨테이너의 변경 사항

Image Layer 3 ← 읽기 전용

Image Layer 2 ← 읽기 전용

Image Layer 1 ← 읽기 전용

[Container Writable layer]

실행 중인 컨테이너 최상단에 생성되는 읽기/쓰기 공간이다.

원본 이미지 레이어는 보호한 채, 파일 수정, 생성 등 모든 변경 사항을 이곳에 기록한다.

컨테이너가 삭제되면 이 레이어의 데이터도 함께 사라진다.

OverlayFS가 여러 Layer를 하나의 파일시스템처럼 합쳐 보여준다.

lowerdir → Image의 읽기 전용 Layer

upperdir → 컨테이너의 쓰기 Layer

merged → 컨테이너가 실제로 보는 합쳐진 파일시스템

workdir → OverlayFS 내부 작업 공간

컨테이너 프로세스는 merged를 자신의 루트 파일시스템으로 보게 된다.

[파일을 수정하면?]

Image Layer에 있는 파일은 읽기 전용이다.

컨테이너가 파일을 수정하면 OverlayFS가 해당 파일을 쓰기 Layer로 복사한 다음 수정한다.

[수정 전]

Image Layer

└─ /etc/nginx/nginx.conf

Container Layer

└─ 비어 있음

[수정 후]

Image Layer

└─ /etc/nginx/nginx.conf ← 원본 유지

Container Layer

└─ /etc/nginx/nginx.conf ← 복사 후 수정

이 동작을 OverlayFS의 **copy-up**이라고 한다.

여기서 주의할 점은 프로세스 fork()의 메모리 CoW와 목적은 비슷하지만 다른 기능이라는 것이다.

fork()의 CoW

→ 메모리 페이지 복사 지연

OverlayFS copy-up

→ 이미지 파일을 쓰기 Layer로 복사

[컨테이너를 삭제하면?]

컨테이너의 writable layer도 일반적으로 함께 사라진다.

```
docker rm web
```

따라서 중요한 데이터는 Volume이나 Bind Mount에 저장해야 한다.

```
docker run -v mydata:/var/lib/mysql mysql
```

Container RootFS

└ 삭제될 수 있는 Writable Layer

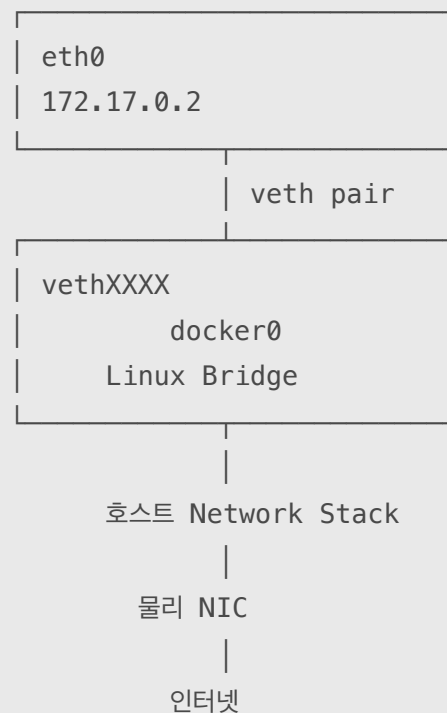
Docker Volume

└ 컨테이너와 별도로 유지되는 데이터

[6] Docker 네트워크

기본적인 bridge 네트워크를 사용하면 Docker는 대략 다음 구조를 만든다.

컨테이너 Network Namespace



[브리지 네트워크]

둘 이상의 네트워크 세그먼트나 장비를 연결하여 하나의 논리적 네트워크로 묶어주는 통신망 연결 방식 또는 장비이다.

OSI 모델의 데이터 링크 계층(2계층)에서 기기의 MAC 주소를 기반으로 동작하며, 물리적 네트워크뿐만 아니라 도커(Docker)나 가상머신(VM)같은 가상화 환경에서도 독립된 네트워크를 상호 연결할 때 광범위하게 쓰인다.

[veth pair]

veth pair는 서로 연결된 가상 네트워크 인터페이스 두 개다.

한쪽으로 패킷 입력
↓
반대쪽으로 패킷 출력

veth 한쪽 끝 ← 가상 랜선 → veth 다른 쪽 끝
호스트에 위치 컨테이너에 위치

Docker는 한쪽을 컨테이너의 Network Namespace에 넣고 이름을 eth0로 변경한다.

veth pair 생성

vethA ↔ vethB
| |
| └ 호스트에 남음: vethXXXX
|
└ 컨테이너로 이동 후: eth0

따라서 컨테이너의 eth0도 본래는 veth 장치다.

컨테이너 쪽 veth는 보통 eth0라는 이름으로 보인다.

컨테이너 eth0

≠ 실제 랜카드

컨테이너 eth0

= veth pair의 컨테이너 쪽 인터페이스

리눅스 커널에서 제공하는 가상 네트워크 인터페이스로, 항상 한 쌍(두 개)이 서로 연결된 형태로 생성된다. 한쪽 인터페이스로 들어오거나 전달된 패킷은 자동으로 다른쪽 인터페이스로 전달되는 가상 케이블 역할을 한다.

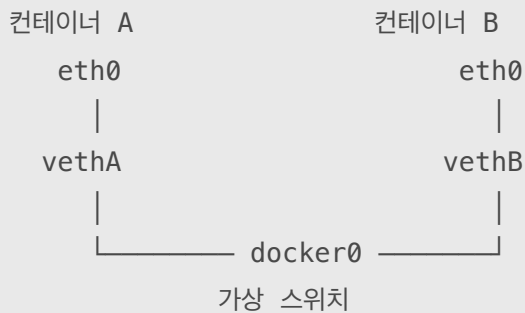
- 양방향 통신: 쌍을 이루는 두 인터페이스(예: veth0, veth1) 중 한쪽에 데이터를 보내면 다른 쪽에서 즉시 받는다.
- 네트워크 네임스페이스 연결: 서로 격리된 네트워크 공간(Namespace)들을 서로 연결하거나, 호스트와 컨테이너를 연결하는 다리 역할을 한다.
- 컨테이너 네트워킹의 기초: 도커(docker)나 쿠버네티스(Kubernetes) 같은 가상화 기술에서 컨테이너 내부와 외부(호스트 브리지)를 연결할 때 핵심 기술로 쓰인다.

[동작 방식]

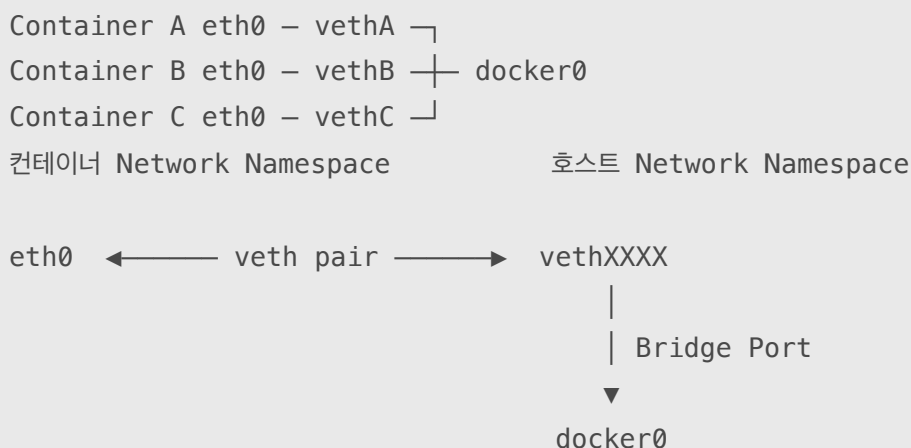
1. 격리된 두 개의 네트워크 네임스페이스가 있다.
2. veth pair를 생성하면 두 개의 가상 랜카드(A, B)가 만들어진다.
3. 랜카드 A는 호스트(또는 브리지)에 두고, 랜카드 B는 컨테이너 내부 네임스페이스로 이동시킨다.
4. 컨테이너가 외부와 통신할 때 랜카드 B로 패킷을 보내면, 연결된 랜카드 A를 거쳐 호스트 네트워크로 전달된다.

[docker0]

docker0는 여러 컨테이너의 호스트 측 veth를 연결하는 가상 L2 스위치인 Linux Bridge이다.



docker0는 MAC 주소를 학습해 패킷을 적절한 veth 포트에 전달한다.



리눅스 브리지

리눅스 커널 안에서 작동하는 가상 L2(데이터 링크 계층) 네트워크 스위치이다.

여러 개의 물리 또는 가상 네트워크 인터페이스를 하나로 묶어, 마치 실제 스위치처럼 MAC 주소를 기반으로 패킷을 주고받도록 돕는다.

[컨테이너끼리 통신할 때]

예를 들어,

컨테이너 A: 172.17.0.2

컨테이너 B: 172.17.0.3

docker0: 172.17.0.1

통신 흐름은 다음과 같다.

컨테이너 A eth0

↓

veth pair

↓

호스트 vethA

↓

docker0

↓

호스트 vethB

↓

veth pair

↓

컨테이너 B eth0

같은 서브넷이기 때문에 docker0가 가상 스위치처럼 두 컨테이너를 연결한다.

[외부 인터넷과 통신할 때]

여기서 호스트의 물리 NIC인 eth0가 등장한다.

```
컨테이너 eth0
    ↓
호스트 veth
    ↓
docker0
    ↓
호스트의 IP 라우팅
    ↓
NAT(MASQUERADE)
    ↓
호스트 물리 NIC eth0
    ↓
외부 인터넷
```

docker0가 물리 eth0와 직접 랜선처럼 연결된 것이 아니다.

```
docker0
    ↓
Linux Kernel의 IP Routing
    ↓
iptables 또는 nftables의 NAT
    ↓
호스트 물리 NIC
```

[외부에서 컨테이너로 들어오는 과정]

다음 명령을 실행했다고 해보자.

```
docker run -p 8080:80 nginx
```

의미는:

호스트 8080번 포트

↓

컨테이너 80번 포트

외부 패킷의 흐름을 지금까지 배운 NIC와 IRQ까지 연결하면 다음과 같다.

- ① 외부 클라이언트가 호스트:8080으로 패킷 전송
- ↓
- ② 물리 NIC가 패킷 수신
- ↓
- ③ RSS가 RX Queue 선택
- ↓
- ④ DMA로 패킷을 RAM에 저장
- ↓
- ⑤ MSI-X 인터럽트 발생
- ↓
- ⑥ Hard IRQ Handler
- ↓
- ⑦ NAPI(인터럽트 + 폴링) / NET_RX_SOFTIRQ
- ↓
- ⑧ 호스트 TCP/IP Stack
- ↓
- ⑨ Netfilter의 NAT·포트 전달 처리
- ↓
- ⑩ docker0와 veth
- ↓
- ⑪ 컨테이너 Network Namespace의 eth0
- ↓
- ⑫ 컨테이너의 80번 Socket
- ↓
- ⑬ nginx 프로세스

즉, 컨테이너가 물리 NIC 인터럽트를 직접 처리하는 게 아니다.

물리 NIC, DMA, MSI-X, Hard IRQ, SoftIRQ는 호스트 커널이 처리하고,

처리된 패킷이 가상 네트워크 장치를 통해 컨테이너의 Network Namespace로 전달된다.

[7] Capability와 Seccomp

컨테이너 내부 프로세스가 root라고 해도 호스트 root의 모든 권한을 그대로 주면 위험하다. 그래서 리눅스의 root 권한을 여러 조각으로 나눈 **Capability**를 사용해.

기존 root

= 모든 관리자 권한

Capability

= 관리자 권한을 여러 조각으로 분리

예:

Capability	역할
CAP_NET_BIND_SERVICE	1024 미만의 포트에 bind
CAP_NET_ADMIN	네트워크 설정 변경
CAP_SYS_ADMIN	마운트 등 광범위한 관리 작업
CAP_CHOWN	파일 소유자 변경
CAP_KILL	다른 프로세스에 Signal 전송

Docker는 기본적으로 필요하지 않은 Capability를 제거한다.

```
docker run --cap-drop=ALL nginx
```

필요한 Capability만 다시 추가할 수도 있다.

```
docker run \  
  --cap-drop=ALL \  
  --cap-add=NET_BIND_SERVICE \  
  nginx
```

[Seccomp (secure Computing Mode)]

리눅스 커널에서 실행 중인 프로세스가 호출할 수 있는 시스템 콜을 제한하여 샌드박싱과 보안 격리를 제공하는 커널 보안 기능이다. 불필요하거나 위험한 시스템 콜을 차단해 공격 표면을 줄인다.

컨테이너 프로세스

| system call



Seccomp Filter

|

├ 허용 → Linux Kernel

└ 차단 → 오류 또는 프로세스 종료

컨테이너의 시스템 콜은 별도의 게스트 커널로 가는 게 아니라 **호스트 커널로 직접 진입**한다.

컨테이너의 `read()`

↓

호스트 Linux Kernel의 `read` 시스템 콜 처리

그래서 시스템 콜 제한이 보안상 중요하다.

AppArmor나 SELinux 같은 LSM도 파일, 프로세스, 장치 접근을 추가로 제한할 수 있다.

[8] docker run 전체 실행 과정

다음 명령을 실행한다고 해보자.

```
docker run -d \  
  --name web \  
  --cpus="1" \  
  --memory="256m" \  
  -p 8080:80 \  
  nginx
```

[1단계] Docker CLI가 요청 전송

docker CLI

|

| 컨테이너 생성 요청

▼

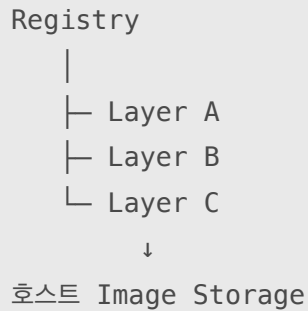
dockerd

요청에는 다음 정보가 들어간다.

```
Image: nginx  
Name: web  
CPU: 1  
Memory: 256MiB  
Port: Host 8080 → Container 80  
Background: true
```

[2단계] Image 확인

Docker daemon은 로컬에 nginx Image가 있는지 확인한다.
없으면 Registry에서 Image Layer를 다운로드한다.



[3단계] 컨테이너 파일시스템 생성

읽기 전용 Image Layer 위에 컨테이너 전용 쓰기 Layer를 만들고 OverlayFS로 합친다.



[4단계] Namespace 생성

컨테이너 프로세스를 위해 다음 격리 공간을 준비한다.

PID Namespace
Mount Namespace
Network Namespace
UTS Namespace
IPC Namespace
Cgroup Namespace
선택적으로 User Namespace

[5단계] cgroup 설정

CPU 사용량 제한
Memory 256MiB 제한
프로세스 수 제한
I/O 관련 정책

그리고 앞으로 실행될 Nginx 프로세스를 해당 cgroup에 소속시킨다.

[6단계] 네트워크 설정

Network Namespace 생성

↓

veth pair 생성

↓

한쪽은 컨테이너에 eth0로 이동

↓

다른 한쪽은 docker0에 연결

↓

컨테이너 IP와 Route 설정

↓

호스트 8080 → 컨테이너 80 포트 전달 설정

[7단계] 보안 정책 적용

Capability 제한

Seccomp Filter 적용

AppArmor/SELinux 정책 적용

읽기 전용·마스킹 경로 설정

[8단계] 컨테이너 프로세스 실행

준비된 Namespace와 RootFS 안에서 Image의 ENTRYPOINT 또는 CMD를 실행한다.

```
execve("/usr/sbin/nginx", ...)
```

그 결과 Nginx가 컨테이너 내부 PID 1이 돼.

컨테이너 내부

PID 1 → nginx

호스트

PID 3150 → 같은 nginx

이 순간부터 컨테이너가 실행 중이라고 표현하지만, 실제로는 격리된 Nginx 프로세스가 실행 중인 것이다.

[9] 컨테이너 실행 중 CPU와 메모리 동작

Nginx가 요청을 처리하면 일반 리눅스 프로세스와 동일하게 동작한다.

```
Nginx
  | read(), write(), epoll_wait()
  ▼
System Call
  ▼
호스트 Linux Kernel
  ├── 스케줄러
  ├── 가상 메모리
  ├── VFS
  ├── TCP/IP Stack
  └── 장치 드라이버
```

[CPU]

호스트 스케줄러는 컨테이너 프로세스를 일반 태스크로 관리한다.

```
CPU Run Queue
  ├── Host Process
  ├── Container A Process
  └── Container B Process
```

cgroup이 사용 가능한 CPU 시간과 가중치를 제한할 뿐이다.

[메모리]

컨테이너 프로세스도 다음 구조를 가진다.

```
task_struct
  |
  └── mm_struct
        ├── VMA: 코드
        ├── VMA: 데이터
        ├── VMA: Heap
        └── VMA: Stack
```

페이지 테이블과 물리 페이지는 호스트 커널이 관리하고, 메모리 사용량이 해당 컨테이너의 cgroup에 계산된다.

[파일]

```
Nginx의 read()  
  ↓  
VFS  
  ↓  
OverlayFS 또는 Volume  
  ↓  
호스트 파일시스템  
  ↓  
NVMe 드라이버  
  ↓  
PCIe/NVMe SSD
```

[네트워크]

```
Nginx의 send()  
  ↓  
컨테이너 TCP/IP Stack  
  ↓  
컨테이너 eth0  
  ↓  
veth pair  
  ↓  
docker0  
  ↓  
호스트 Network Stack  
  ↓  
물리 NIC
```

‘컨테이너 TCP/IP Stack’이라고 표현하지만 별도 커널이 있는 것은 아니다.
같은 호스트 커널의 네트워크 스택이 컨테이너의 Network Namespace 정보를 기준으로 처리하는 것이다.

[10] 컨테이너 종료

다음 명령을 실행하면:

```
docker stop web
```

일반적으로 Docker는 컨테이너의 PID 1에 종료 Signal을 보낸다.

```
dockerd
|
| SIGTERM
▼
컨테이너 PID 1: nginx
```

Nginx가 제한 시간 내에 종료하지 않으면 강제 종료 Signal을 보낼 수 있다.

```
SIGTERM
↓ 일정 시간 기다림
SIGKILL
```

프로세스가 종료되면:

- Namespace 관련 자원이 정리됨
- cgroup에서 프로세스가 사라짐
- veth가 제거됨
- 컨테이너 writable layer는 컨테이너가 삭제될 때 제거됨
- Volume은 별도로 삭제하지 않는 한 유지될 수 있음

docker stop은 컨테이너 실행을 멈추는 것이고, docker rm은 컨테이너 객체와 writable layer를 제거하는 것이다.

[11] Namespace와 cgroup만 있으면 Docker인가?

아니다.

Docker는 이러한 리눅스 기능을 조합하고 사용하기 쉽게 관리하는 플랫폼이다.

```
Linux Kernel
├─ Namespace
├─ cgroup
├─ OverlayFS
├─ veth
├─ bridge
├─ Netfilter
├─ Capability
├─ Seccomp
└─ LSM
```

Docker

- └ Image 형식과 Layer 관리
- └ Registry 연동
- └ Container 수명주기
- └ Network 자동 구성
- └ Volume 관리
- └ 빌드 기능
- └ 사용자 CLI와 API 제공

컨테이너 격리 기능을 실제로 제공하는 것은 리눅스 커널이고,
Docker는 그 기능을 조합해서 이미지 배포와 컨테이너 실행을 편리하게 만들어 주는 관리 도구다.

[12] Docker 컨테이너의 실체

최종적으로 Docker 컨테이너의 실체는 다음과 같다.

컨테이너

- = Namespace로 보이는 자원을 격리한 프로세스
- + cgroup으로 자원 사용량을 제한한 프로세스
- + OverlayFS로 독립된 RootFS를 제공받은 프로세스
- + veth와 Network Namespace를 제공받은 프로세스
- + Capability·Seccomp·LSM으로 권한을 제한한 프로세스

[한 문장 정리]

**Docker는 별도의 운영체제를 만드는 것이 아니라,
호스트 리눅스 커널의 Namespace로 프로세스를 격리하고, cgroup으로 자원을 제한하며,
OverlayFS로 독립적인 파일시스템을 제공하고,
veth·Bridge로 가상 네트워크를 구성하여 하나의 독립된 시스템처럼 보이게 만든다.**

전체 실행 흐름도 한 줄로 연결하면 다음과 같다.

- docker run
- dockerd
- containerd
- runc
- Namespace 생성
- cgroup 설정
- OverlayFS RootFS 준비
- veth·Bridge 연결
- 보안 정책 적용
- execve()
- 격리된 일반 Linux 프로세스